Thesis Proposal for the Degree of Master of Science in Computer Science

Reducing False Alarms in IoT Networks: A Deep Learning Based Intrusion Detection System

Jinesh Subedi
(2021-1-23-0008)

Nepal College Of Information Technology
Faculty of Science and Technology
Pokhara University, Nepal

Nov, 2025

Table of content

List of Tables

List of Figures

List of Abbreviation/Acronyms

ADYSN	Adaptive Synthetic Sampling
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Networks
FNR	False Negative Rate
FPR	False Positive Rate
GRU	Gated Recurrent Unit
MITM	Man-in-the-Middle
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique

CHAPTER 1

INTRODUCTION

The Internet of Things (IoT) has changed modern life by connecting billions of devices and are now present in every aspect of our daily lives. Examples range from our homes, with smart appliances and entertainment systems, to smart cities and their hidden infrastructure, like pedestrian and road sensors. The smart grid, self-driving car systems, smart medical devices, industrial control systems, and robotics are just some areas where IoT devices are used every day. The data collected by these devices needs storage and analysis. To gain insights from this data and support smart applications, methods like machine learning (ML) are commonly used. Detecting cyber attacks is part of these smart applications. With the ongoing rise in cyber attacks targeting IoT systems, monitoring this large amount of data for attack detection is essential. However, this can only be done through automated methods that rely on ML and deep learning (DL). DL is a subset of ML that has gained popularity in various fields, including science, finance, medicine, and engineering. Using DL for intrusion detection has also become more common because it allows for deeper analysis of network traffic and more accurate identification of anomalies compared to traditional ML methods. This unprecedented scale and heterogeneity, however, have simultaneously created an enormous and vulnerable attack surface. These low-power systems are constantly exposed to a spectrum of sophisticated cyber threats, such as Distributed Denial-of-Service (DDoS), Man-in-the-Middle (MITM), data injection, and ransomware attacks.

In these networks, intrusion detection systems (IDS) are a crucial defense mechanism because they can identify unusual activity that traditional security measures would miss. These security mechanisms monitor network traffic and device behavior to identify malicious activity that traditional perimeter defenses, like firewalls, often fail to detect. Recently, deep learning techniques have emerged as the most promising solutions, reporting superior overall detection accuracy (often above 98%) compared to traditional machine learning classifiers. Nevertheless, despite the tremendous advancements, the high false positive rate (FPR) remains one of the most significant present issues with IoT IDS.

Even though state-of-the-art models like CNN-GRU [1] and Random Forest [2] have reported high overall detection accuracy of more than 98%, their FPRs, ranging between 0.50% and 0.75%, still equate to a large number of false alarms.

In reality, in large IoT installations, even a low FPR can result in thousands of false alarms per day, causing resource waste, accelerated battery drain, network congestion, and analyst fatigue. Furthermore, frequent false positives undermine operators' trust in automatic intrusion detection systems, leading them to disregard warnings or completely disable detection systems, increasing their vulnerability to successful cyberattacks.

To address these critical challenges, this research introduces a novel optimized Bidirectional Long Short-Term Memory (BiLSTM) architecture specifically designed for

IoT intrusion detection, with particular emphasis on reducing false positive rates while maintaining high detection sensitivity across diverse attack categories.

1.1 Background

The security landscape for the Internet of Things (IoT) presents significant challenges. Traditional Intrusion Detection System (IDS) models are often unsuitable because of the demanding computational and energy constraints inherent to most IoT devices [4]. Furthermore, the heterogeneous nature of IoT traffic—which includes everything from routine sensor telemetry to malicious data like botnet attacks—differs substantially from typical network data [3]. This variability makes it difficult to create security models with good generalization, leading to poor performance when trying to identify diverse threats.

Current research favors Deep Learning and Hierarchical Models for improved IoT security. Deep learning often utilizes hybrid models like CNN-GRU [1], RNN-GRU [12], and LSTM-GRU [10], which are highly effective at extracting complex features from diverse IoT data. However, a limitation of these models is their reduced capability in detecting bidirectional temporal patterns within the data. Alternatively, Hierarchical Models have shown promise, with one study demonstrating that Multi-Layer Perceptrons (MLPs) significantly reduced false alarms by 87.52%. Despite this success, the general applicability of hierarchical models is limited by a lack of IoT-specific validation in their deployment and testing environments[7].

This proposal combines the sequence learning capability of Bi-LSTM with SHAP-based feature optimization to reduce false alarms and improve interpretability of the model. In addition to efficiency, lowering FPR in IoT IDS systems is critical for security, reliability, and sustainability.

1.2 Statement of the problem

The high false positive rate (FPR) remains a critical challenge for Internet of Things (IoT) intrusion detection systems (IDS), significantly diminishing their real-world efficacy despite advancements in their underlying models. Even systems boasting high detection accuracy, often over 98%, frequently struggle to adapt to the dynamic, heterogeneous, and ever-changing nature of IoT network traffic. This varied traffic, originating from a multitude of devices, protocols, and behavioral patterns—many of which are benign but unique—can easily be misclassified as malicious anomalies by the anomaly-based detection techniques commonly employed in IoT IDS. This results in a continuous flood of false alarms, which leads to alert fatigue among security analysts, making it difficult to differentiate genuine threats from harmless events, and consequently increasing the risk of overlooking a true attack. [1][2]

1.3 Research objectives

To perform a thorough evaluation of both baseline and enhanced BiLSTM models to measure improvements in intrusion detection effectiveness across various aspects.

To Develop an optimized intrusion detection system that achieves class-specific performance targets, with particular focus on reducing false alarms while maintaining high detection accuracy for all attack types.

1.4 Significance/Rationale of the study

This study presents a Bi-LSTM framework to address the important problem of high false positive rates (FPR) in IoT intrusion detection systems (IDS). Because they rely on non-IoT datasets and unidirectional learning, traditional IDS models like CNN-GRU and Random Forest overproduce false alarms that waste resources, overwhelm analysts, and delay adoption. In addition to efficiency, sustainability, security, and reliability all depend on lowering FPR in IoT IDS systems.

CHAPTER 2

LITERATURE REVIEW

2.1 Related papers

Moustafa introduced the ToN-IoT dataset, a comprehensive benchmark for evaluating intrusion detection systems (IDS) in Internet of Things (IoT) and Industrial IoT (IIoT) networks. By merging information from multiple sources, such as device telemetry, operating system logs, and network traffic, the dataset captures the diversity of actual IoT systems. In his comparison of traditional machine learning and deep learning techniques, Random Forest (RF), Classification and Regression Trees (CART), and LSTM, against a variety of IoT-specific attacks, Moustafa came to the conclusion that RF and CART outperformed LSTM in multi-class classification. To enhance transparency and efficiency in intrusion detection system (IDS) solutions, the study notes that while ToN-IoT effectively bridges the gap of real IoT datasets, more testing with modern deep learning techniques, such as Bi-LSTM, and the use of explainable AI (XAI) techniques are necessary[9].

Using feature importance analysis in binary and multi-class classification frameworks, Rana and Singh proposed a Random Forest-based IDS with a 99.4% accuracy and decreased the false alarm rate to 0.50% on the NSL-KDD dataset. Even though it works well for conventional networks, using a non-IoT dataset makes the model unscalable because it doesn't have any attack patterns specific to IoT and has a high failure rate when used in IoT environments [2].

Mijalkovic and Spognardi concentrated on the application of deep learning and artificial neural networks (ANN) and deep neural networks (DNN) for minimizing the false negative rates (FNR) of intrusion detection systems (IDS). They addressed the problems of minority-class attack detection, especially uncommon Internet of Things-specific attacks. In their approach, they experimented with imbalanced learning techniques on datasets such as UNSW-NB15 (non-IoT) and NSL-KDD and utilized DNN and ANN for multi-class classification. Similar to IoT-specific attacks, the minority class recall of the experiments was low and their generalization to IoT traffic was poor, despite having an improvement in their FNR. Use of Non-IoT dataset and not prioritizing the minimization of FPR were two of the significant drawbacks [3].

By combining temporal sequence modeling and spatial feature extraction, Henry et al. developed a CNN-GRU-based intrusion detection system (IDS) that demonstrated 98.73% accuracy and a low FPR of 0.0075 on the CICIDS-2017 dataset. Aside from its limited generalizability to IoT networks due to its evaluation on a non-IoT dataset, the model's two main shortcomings are its inability to capture long-term dependencies and its imbalanced attack classification with few attack classes. [1].
Siganos et al.enhanced analyst confidence in model outputs by integrating SHAP-based Explainable AI (XAI) with an Artificial Intelligence (AI)-driven Intrusion Detection

System (IDS) for the Internet of Things. They acknowledged current challenges such as false alarms and greater explainability. They evaluated on IEC 60870-5-104 protocol data (industrial IoT) and the solution emphasized using SHAP for visualization of feature importance. Although decision-making was facilitated by Explainable Artificial Intelligence (XAI), no impact was observed on false positive rates (FPR), and protocol-specific anomalies led to periodic false alarms. The study was limited by the exclusive consideration of a single industrial protocol and the absence of mechanisms for reducing false alarms. The results highlighted the imperative to incorporate XAI in Internet of Things Intrusion Detection Systems (IoT IDS) for mitigating FPR issues [5].

On the basis of a two-stage MLP model, Moore et al. proposed an IDS that achieved 73.44% recall, 0.59% FPR, and a 87.52% decrease in false alarms on the CIDDS-001 dataset. The high-risk alerts were prioritized, and minority-class detection was improved through multi-task learning. Although the model was successful in decreasing false positives, its application was restricted to IoT environments as it was evaluated solely on non-IoT datasets. Its low performance against the attacks that are uncommon further indicates that it requires additional fine-tuning to deal with Internet of Things challenges [7].

By combining Bi-LSTM to enable bidirectional temporal modeling with SHAP and LIME for interpretability, Sivamohan and Sridhar created a BiLSTM-XAI model that produced a 98.2% accuracy rate and a 0% false positive rate (FPR) on the NSL-KDD and Honeypot datasets. The suggested model improved feature selection by utilizing Krill Herd Optimization (KHO). Despite the impressive results, the model's applicability to the current IoT scenario is questionable because it was tested on outdated, non-IoT datasets rather than being compared to the current, IoT-specific attack patterns. Further testing on actual IoT traffic is necessary to determine its feasibility in real-world scenarios. [8].

Using the ToN-IoT dataset, Gad et al. proposed a distributed IDS using XGBoost for binary and multi-class classification of IoT-based attacks. The study applied the Synthetic Minority Over-sampling Technique (SMOTE) for handling class imbalance and feature selection using the Chi-squared (Chi^2) approach to reduce the feature set to 20 features, successfully mitigating serious issues related to data quality. The training efficacy and classification performance were greatly improved by tuning the preprocessing pipeline. When testing generalization to other attack types such as Ransomware and DDoS, the optimized model performed significantly better than other machine learning baselines, with higher accuracy and F1-scores.
However, using a traditional machine learning model like XGBoost compromises the system's capacity to recognize complex spatial-temporal patterns associated with IoT traffic. Furthermore, the study does not incorporate explainability frameworks and deep learning techniques, which are increasingly important in contemporary IDS research. The model would be enhanced by incorporating robust temporal modeling and interpretability capabilities, despite its effectiveness in detecting anomalies in IoT environments [11].

Using the CICIDS-2017 dataset, Al-Kahtani et al. proposed a hybrid LSTM-GRU model in conjunction with a PSO-GA (Particle Swarm Optimization–Genetic Algorithm) feature selection technique, which achieved 98.86% accuracy. Their approach successfully uses deep sequential models to achieve robust attack detection and evolutionary algorithms to choose the best subset of features. The model is computationally costly and has not been tested on IoT-specific traffic, which makes it unsuitable for deployment on resource-constrained IoT devices, even though it achieves state-of-the-art performance in traditional network environments. Consequently, the framework is not directly appropriate for the requirements of IoT security, even though it excels at feature selection and detection [10].

A hybrid RNN-GRU approach to IoT intrusion detection was proposed by Khan et al. and outperformed state-of-the-art methods, achieving 99% accuracy on TON-IoT network flow data. With extensive data engineering to fine-tune features, their method used GRU for long-term dependency capturing and RNN for sequence modeling. With 99% accuracy for the network flow and 98% accuracy for the application layer, the model outperformed cutting-edge methods in identifying frequent IoT attacks. However, two significant limitations of the TON-IoT dataset are its high computational requirements for devices with limited resources and its limited representation of real-world attack vectors. The hybrid model exhibits promise despite these obstacles, but more testing against various attack scenarios is necessary[12].

Using the TON-IoT dataset, Alabbadi & Bajaber combined Explainable AI (XAI) methods such as SHAP and LIME with 1D-CNNs and DNNs to detect IoT intrusions with 99.24% accuracy. Their method showed how XAI can improve IoT attack classification transparency. The strategy used pre-trained TabNet for effective tabular data processing and 1D-CNNs for feature extraction from IoT data streams. For specific attack classes, the results showed 100% accuracy for CNN/DNN, while SHAP/LIME provided useful feature importance insights. However, the study had certain shortcomings, such as a lack of examination of adversarial robustness for XAI outputs and a restricted emphasis on minimizing the false positive rate (FPR). The model is useful for detecting IoT attacks, but it is not FPR- and security-optimized, according to the conclusion [4].

2.2 Research Gap

- Most IDS are trained on non-IoT datasets (CICIDS-2017, NSL-KDD), existing IDS models [1][2] have poor generalization to IoT traffic patterns (e.g., botnets, DDoS on constrained devices).
- State-of-the-art models [1][2] report FPRs of 0.50 to 0.75%; however, in IoT environments, these rates increase because of noise and heterogeneous traffic.
- Previous research ignores cost-sensitive tuning to minimize FPR under stringent FNR constraints (e.g., ≤1%) in favor of accuracy or FNR reduction [3].

2.3 Contribution

This study introduces an intrusion detection system that leverages a Bidirectional LSTM (Bi-LSTM) to autonomously learn spatiotemporal patterns from network traffic sequences. Our method integrates automated data balancing, Bayesian hyperparameter optimization, and dynamic threshold tuning to simultaneously achieve two critical objectives: a reduction in false alarm rates by over 50% and the maintenance of a false negative rate below 1% across all threat categories.

CHAPTER 3

RESEARCH METHODOLOGY

The proposed Bi-LSTM-Based IDS for IoT Networks is to develop and test a Bi-LSTM intrusion-detection system for IoT networks, including dataset selection, feature engineering, normalization, model building, training, hyperparameter tuning, monitoring, and evaluation.

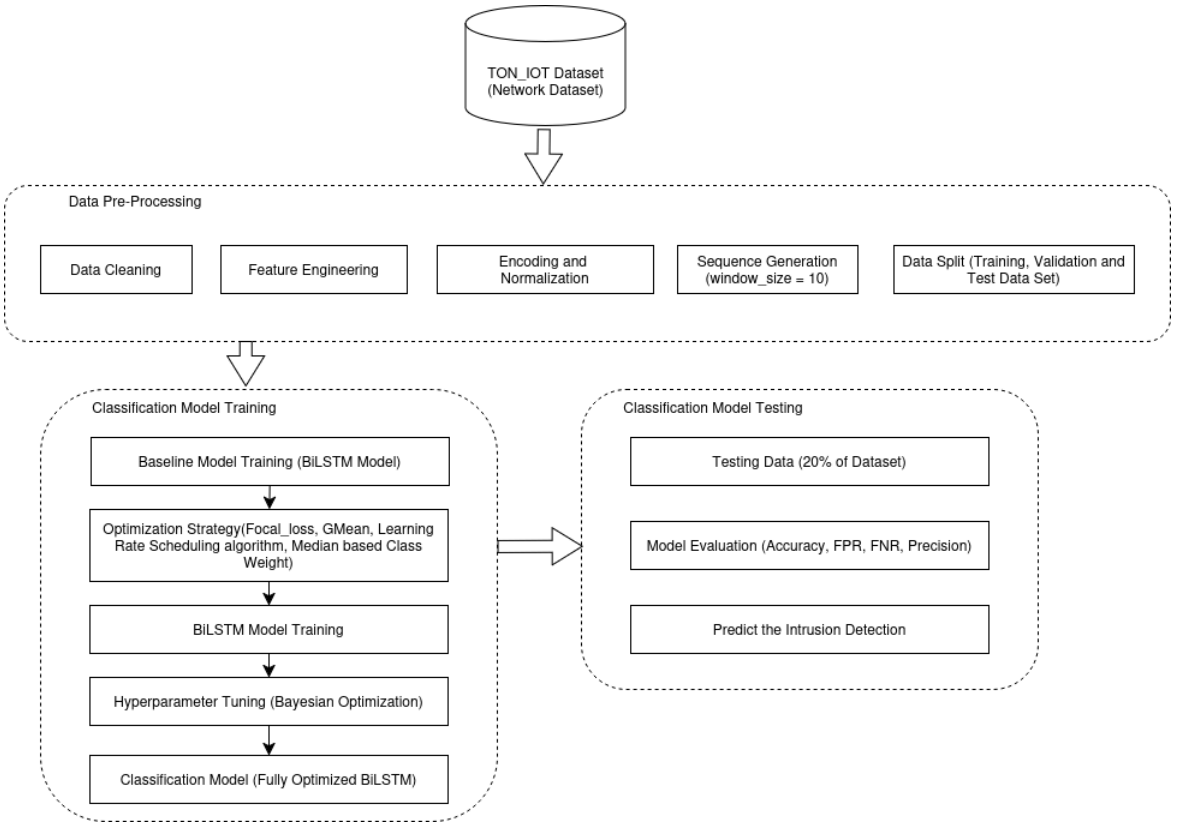


Fig. 3.1 : Block Diagram of Optimized IOT IDS

3.1 Data Collection and Cleaning

UNSW Canberra developed the comprehensive benchmark known as the TON_IoT dataset [9], which combines data from various IoT and IIoT sources. Because TON_IoT was created especially for IoT/IIoT ecosystems, it is more applicable to contemporary cyber-physical systems (smart factories, smart homes). On the other hand, CICIDS2017 and NSL-KDD are more concerned with conventional IT networks. Despite being IOT-specific, N-BaIoT only targets a few attack classes, primarily Mirai and Bashlite botnets. TON_IOT includes a broad range of attack types, including ransomware, injection, DoS, DDoS, backdoor, MITM, password attacks, and more.

Table 3.1: All Network Data

Type	**All network Data**	**Data Cleaning**	**Training and Testing**
backdoor	508116	505385	20000
ddos	6165008	6082893	20000
dos	3375328	1815909	20000
injection	452659	452659	20000
mitm	1052	1043	1043
password	1718568	1718568	20000
ransomware	72805	32214	20000
scanning	7140161	7140158	20000
xss	2108944	2108944	20000
normal	796380	689279	300000

Nour Moustafa deliberately provided a curated train/test subset of the TON_IoT network data for benchmarking. This subset contains about 461,043 flows in total (300,000 normal + 161,043 attack) Crucially, *each attack category was sampled to roughly the same size*: 20,000 flows for Backdoor, DDoS, DoS, Injection, Password, Ransomware, Scanning and XSS (and the 1,043 available flows for MITM). In other words, the authors capped each major attack class at 20k examples (with normal traffic 300k). [9][13]

Table 3.2: Dataset Feature Counts

Network Features	**Features (45 features)**
Connection Activity	Ts, src_ip, src_port, dst_ip, dst_port, proto, service, duration, src_bytes, Dst_bytes, conn_state, missed_bytes
Statistical Activity	Src_pkts, src_ip_bytes, dst_pkts, dst_ip_bytes,
DNS Activity	Dns_query, dns_qclass, dns_qtype, dns_rcode, dns_AA, dns_RD, dns_RA, dns_Rejected
SSL Activity	Ssl_version, ssl_cipher, ssl_resumed, ssl_established, ssl_subject, ssl_issuer
HTTP Activity	Http_trans_depth, http_method, http_url, http_version,

	http_request_body_len, http_response_body_len, http_status_code, http_user_agent, http_orig_mime_types, http_resp_mime_types
Violation Activity	Weird_name, weird_addl, weird_notice
Data Labeling	Label (0 indicates normal, 1 for attack); Type (normal, DoS, DDoS, backdoor, etc)

3.3 Feature Engineering and Normalization:

3.3.1 Feature Engineering Process

a. Data Type Identification & Separation

- Categorical Features: Text-based or discrete categorical variables including protocol_type, service, flag, and other nominal attributes that represent distinct classes or states within network communications.
- Numerical Features: Continuous or discrete numerical measurements such as duration, src_bytes, dst_bytes, src_pkts, and other quantitative metrics capturing traffic volume, timing, and size characteristics.

b. Categorical Feature Encoding

- Label Encoder is used to Converts text categories to numerical indices
  Example:
  protocol_type: ['tcp', 'udp', 'icmp'] → [0, 1, 2]
  service: ['http', 'ftp', 'smtp'] → [0, 1, 2]

  Formula:
  $L = \{c_1, c_2, \ldots, c_n\} f(c_i) = i - 1$ (1)

  Then, LabelEncoder assigns:

  $f(c_i) = i - 1, \quad where\ i \in [1, N]$ (2)

  where iii is the index of the class in the sorted unique list.

- One-Hot Encoding converts each integer class label into a binary vector
  Example:
  Normal: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
  backdoor: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]

  Formula:
  $O_i = [0, 0, .. 1, .., 0]$ (3)
  where
  $O_i[j] = \{ 1 , \text{if } j = y_i$ (4)
  $0 , \text{otherwise } \}$

And $O_i \in R^N$.

c. **Target Variable Processing**
   - Binary Classification: Normal (0) vs Attack (1) for fundamental intrusion detection capability
   - Multi-class Classification: Specific attack type categorization (DoS, Backdoor, DDoS, Injection,etc.) for detailed threat analysis and classification

3.3.2 Normalization

In intrusion detection systems (IDS), normalization plays a vital role in transforming heterogeneous numerical features into a comparable scale. Since network traffic data often contains attributes with varying units (e.g., packet size, duration, and byte count), unnormalized features can dominate the learning process and bias the model.
To address this, the **Min-Max Normalization** technique is applied. It rescales each feature to a fixed range — typically **[0, 1]** — ensuring that all input features contribute proportionally during model training. This approach preserves the relative relationships between feature values while maintaining sensitivity to small variations, which is crucial for detecting subtle attack patterns.

Formula

$$x' = (x - x_{min}) / (x_{max} - x_{min}) \quad (5)$$

Where

- x = Original feature value
- x_{min} = Minimum value of the feature in the dataset
- x_{max} = Maximum value of the feature
- x' = Normalized value within the range [0, 1]

3.4 Two-Stage Train-Validation Split Strategy

3.4.1 Phase 1: Initial Holdout Split

The initial holdout split serves as the foundational separation between data used for model development and data reserved exclusively for final evaluation. This phase ensures complete isolation of the test set from any model training or parameter tuning activities, preventing data leakage and providing an unbiased assessment of model generalization.

Implementation Results:

- **Training Set**: 352,834 samples (80% of total dataset)
- **Testing Set**: 88,209 samples (20% of total dataset)
- **Feature Dimension**: 44 features per sample

- **Target Representation**: 9-class multi-class classification format

3.4.2 Phase 2: Post-Sequence Validation Split

This strategic phase occurs after critical preprocessing steps—specifically sequence generation and class balancing—to maintain temporal integrity and address the unique challenges of sequential data in intrusion detection.

Final Data Partitions:

- **Final Training Sequences**: 285,556 sequences (80% of training data)
- **Validation Sequences**: 71,389 sequences (20% of training data)
- **Sequence Dimensions**: 10 timesteps × 44 features per sequence
- **Temporal Structure**: Complete preservation of sequential dependencies

3.5 Sequence Creation and Windowing

To transform the preprocessed tabular data into a format suitable for the Bi-LSTM model's sequential processing capabilities, a temporal windowing technique was implemented. This approach converts individual network events into contextual sequences that capture temporal dependencies essential for effective intrusion detection.

3.5.1 Sliding Window Implementation

A fixed-size sliding window with window_size = 10 timesteps was applied to create sequential contexts from the network traffic data. This parameter was carefully selected to balance computational efficiency with the capture of meaningful temporal patterns in network behavior. The Bi-LSTM to gradually learn patterns and dependencies, this technique is essential.

Given:
Input dataset $X = [x_1, x_2, x_3, \ldots, x_N]$

where each $x_i \in R^d$ (a feature vector of size d).

Corresponding labels $Y = [y_1, y_2, y_3, \ldots, y_N]$

Fixed **window size** w

We generate overlapping sequences and labels such that:

$$X(i) = [x_I, x_{i+1}, \ldots, x_{i+w-1}] \quad \text{for} \quad i = 1, 2, \ldots, N - w + 1 \qquad (6)$$

$$y^{(i)} = y_{i+w-1}$$

Total Number of Generated Sequences

$$n_{seq} = N - w + 1 \quad (7)$$

Training Set Distribution:

- Normal traffic (Class 5): 239,994 sequences (68.0%)
- Attack classes: ~16,000 sequences each (4.5% per class)
- MITM minority (Class 4): 834 sequences (0.24%)

Testing Set Distribution:

- Normal traffic (Class 5): 59,995 sequences (68.0%)
- Attack classes: ~4,000 sequences each (4.5% per class)
- MITM minority (Class 4): 209 sequences (0.24%)

This sequential structuring enables the Bi-LSTM model to learn from the temporal evolution of network traffic, capturing patterns that would be invisible in isolated event analysis. The preserved 3D tensor format (samples, timesteps, features) provides the exact input structure required for subsequent model training and evaluation phases.

3.6 Class Imbalance Analysis

3.6.1 Initial Class Distribution

The dataset exhibits significant class imbalance, which was analyzed after sequencing the time-series data into supervised learning format. The initial class distribution in the training set revealed a highly skewed multi-class problem:

Table 3.3: Initial Class Distribution

Class	Samples	Percentage	Category
Class 5	239,994	68.0%	Majority Class
Class 0	15,999	4.5%	Minority Class
Class 1	16,000	4.5%	Minority Class
Class 2	16,000	4.5%	Minority Class
Class 3	16,000	4.5%	Minority Class
Class 6	15,999	4.5%	Minority Class
Class 7	15,999	4.5%	Minority Class
Class 8	16,000	4.5%	Minority Class

Class 4	834	0.2%	Extreme Minority

Total Samples: 352,825 sequences

3.6.2 Imbalance Severity Assessment

The analysis revealed an extreme imbalance between classes:

- Majority vs Minority Ratio: Class 5 constitutes approximately 68% of the entire dataset
- Extreme Minority Case: Class 4 represents only 0.2% of total samples
- Imbalance Factor: The ratio between majority class (5) and extreme minority class (4) is approximately 287:1

This level of imbalance poses significant challenges for machine learning models, as they tend to be biased toward the majority class, potentially ignoring or misclassifying the underrepresented classes.

3.6.3 Resampling Strategy

3.6.3.1 Targeted Approach

To address the imbalance while maintaining data integrity, a targeted resampling strategy was implemented:

- Selective Oversampling: Only Class 4 was selected for resampling, as other minority classes had sufficient representation (~16,000 samples each)
- Target Sample Size: Class 4 was oversampled from 834 to 5,000 samples
- Preservation of Natural Distribution: The natural distribution of other classes was maintained to avoid unnecessary synthetic data generation

3.6.3.2 Resampling Techniques

- **ADASYN (Adaptive Synthetic Sampling)**
    - Adaptive algorithm that focuses on difficult-to-learn samples
    - Generates more synthetic data near decision boundaries
    - Considers the density distribution of minority classes

  Steps:

  1. Compute the *imbalance ratio* between majority and minority classes.
  2. For each minority instance x_i, compute:

$$r_i = \frac{\Delta_i}{\sum_{j=1}^{n} \Delta_j} \quad (8)$$

Where:

$\Delta_i = number\ of\ majority\ neighbors\ among\ the\ k - nearest\ neighbors\ of\ x_i$

$n = total\ number\ of\ minority\ samples$

3. Determine how many synthetic samples to generate for x_i.

$$g_i = r_i \, x \, G \quad (9)$$

Where $G = total\ number\ of\ synthetic\ samples\ to\ create$

4. For each g_i, generate synthetic samples using the same SMOTE interpolation formula:

$$x_{new} = x_i + \delta * (x_{zi} - x_i) \quad (10)$$

where x_{zi} is a randomly chosen minority neighbor.

3.6.4 Post-Resampling Distribution

After applying the resampling strategy:

Table 3.4: Class Resampling Strategy

Class	Original Samples	Resampled Samples	Change
Class 4	834	5,000	+4,166 (+500%)
All Other Classes	351,991	351,991	No change
Total	352,825	356,991	+4,166

3.7 Classification Model Architecture

The proposed Bidirectional LSTM (Bi-LSTM) architecture is specifically designed to capture comprehensive temporal dependencies in sequential network traffic data by processing information in both forward and backward directions.

3.7.1 Input Layer Configuration:

- **Input Shape**: (10, 44) - representing 10 consecutive timesteps with 44 features per timestep
- **Sequence Length**: Optimized to capture short-term network behavior patterns
- **Feature Dimension**: Comprehensive representation of network traffic characteristics

3.7.2 Hierarchical Bi-LSTM Layers

- First BiLSTM Layer
  - Output Dimension: 508 features (254 forward + 254 backward)
  - Temporal Preservation: Maintains 10 timesteps for hierarchical processing
  - Parameter Count: 607,568 parameters
  - Function: Low-level temporal feature extraction from raw network sequences
- Second BiLSTM Layer
  - Output Dimension: 254 features (127 forward + 127 backward)
  - Sequence Encoding: Summarizes entire sequences into fixed-length representations
  - Parameter Count: 646,176 parameters
  - Function: High-level temporal pattern recognition and sequence encoding.

3.7.3 Advanced Regularization Strategy

- **First Normalization**: Stabilizes 508-dimensional feature distributions across 10 timesteps (1,016 learnable parameters)
- **Second Normalization**: Normalizes 254-dimensional encoded representations (508 learnable parameters)
- **Purpose**: Ensures stable activation distributions and improves training convergence

3.7.4 Dropout Regularization for Prevention of Overfitting:

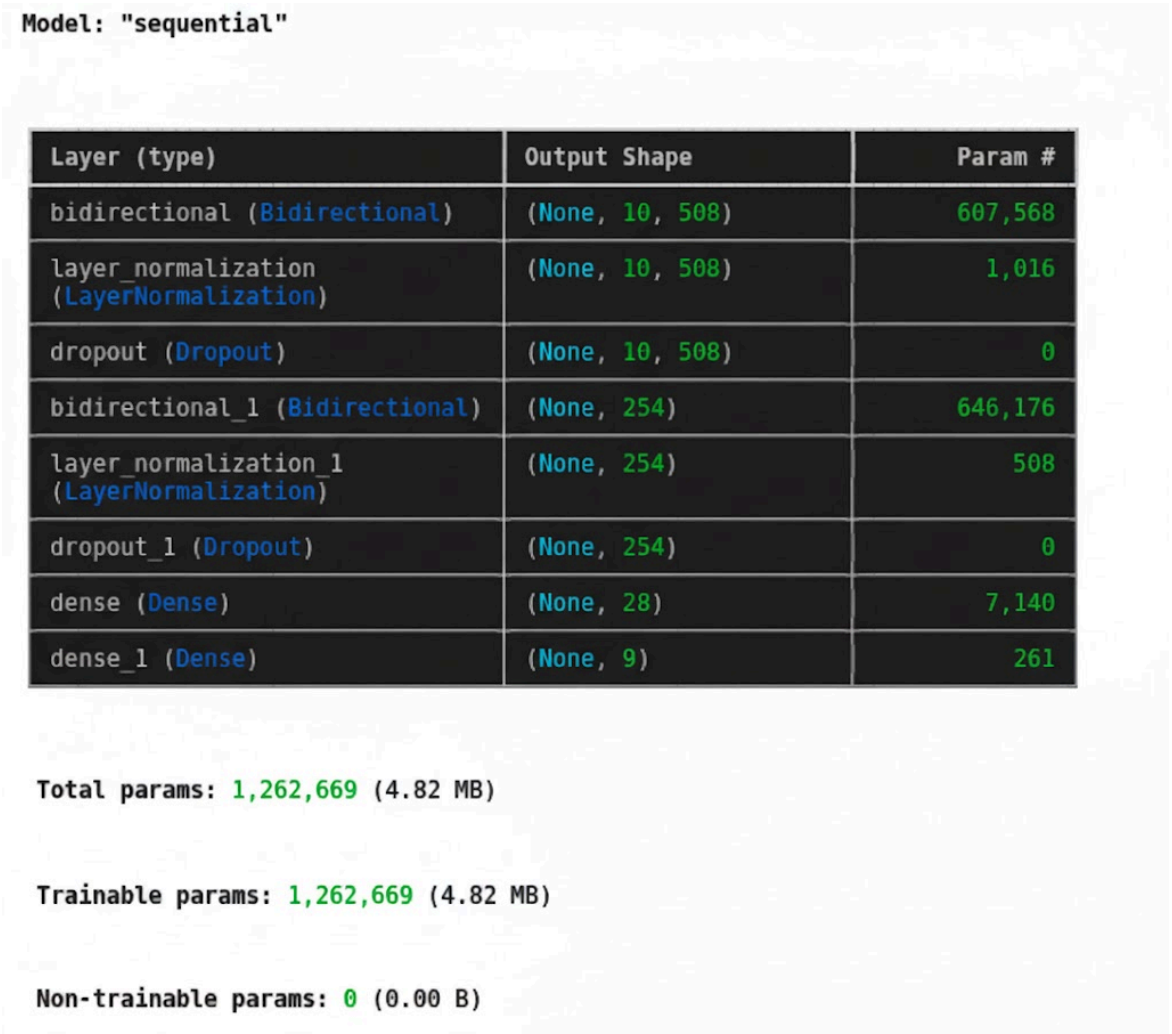
- **Rate**: 0.2 (20% of units randomly deactivated during training)
- **Placement**: Applied after each Bi-LSTM layer
- **Function**: Prevents overfitting and enhances model generalization capability

3.7.5 Fully Connected (Dense) Layer:

- **Dimensionality Reduction**: 254 features → 28 features (89% compression)
- **Parameter Count**: 7,140 trainable parameters
- **Function**: Forces retention of most discriminative features while reducing computational complexity

3.7.6 Output Layer Configuration for Multi-class Classification

- **Output Units:** 9 neurons corresponding to attack categories
- **Activation Function:** Softmax for probability distribution output
- **Parameter Count:** 261 parameters
- **Interpretation:** Each output represents probability of specific attack class



Model: "sequential"

Layer (type)	Output Shape	Param #
bidirectional (Bidirectional)	(None, 10, 508)	607,568
layer_normalization (LayerNormalization)	(None, 10, 508)	1,016
dropout (Dropout)	(None, 10, 508)	0
bidirectional_1 (Bidirectional)	(None, 254)	646,176
layer_normalization_1 (LayerNormalization)	(None, 254)	508
dropout_1 (Dropout)	(None, 254)	0
dense (Dense)	(None, 28)	7,140
dense_1 (Dense)	(None, 9)	261

Total params: 1,262,669 (4.82 MB)

Trainable params: 1,262,669 (4.82 MB)

Non-trainable params: 0 (0.00 B)

Fig. 3.2 : Model Architecture Summary

3.8 Training Strategy Design

3.8.1 Bayesian Optimization Strategy

The hyperparameter tuning process employed Bayesian Optimization through Keras Tuner, utilizing a systematic approach to navigate the complex parameter space of the Bi-LSTM architecture.

- Objective Function: Minimize validation FPR while constraining FNR ≤1%.
- Key Hyperparameters:

- LSTM Unit 1: An integer between 128 and 512.
- LSTM Unit 2: An integer between 64 and 254.
- Hidden Layer Neurons: An integer between 16 and 64
- Dropout Rate: A real value between 0.1 and 0.5.
- Learning Rate: A real value from [0.0001, 0.01] (controls overfitting),
- Gamma: An integer between 1 and 5.
- Alpha: An integer between 0.1 and 0.5.

3.8.2 Optimizer Selection and Configuration

- Adam is an adaptive learning rate optimization algorithm that combines the advantages of two other extensions of stochastic gradient descent
- The learning rate controls how much to update the model weights in response to the estimated error each time the weights are updated.
- Gradient clipping prevents the exploding gradient problem by limiting the magnitude of gradients during backpropagation.

3.8.3 Loss Function Selection

- Categorical Crossentropy
  - Standard loss function for multi-class classification
  - Measures the difference between true probability distribution and predicted distribution
  - CE = -Σ(y_true * log(y_pred))
- Focal Loss
  - Advanced loss function specifically designed for class imbalance
  - Adaptively scales crossentropy based on classification difficulty
  - FL = -Σ(α * (1 - y_pred)^γ * y_true * log(y_pred))

3.8.4 Class Weight Calculation

The class weight calculation employs a median-frequency balancing approach to address dataset imbalance.

Formula: $\text{weight}_c = \frac{\text{median\_frequency}}{\text{frequency}_c}$

3.8.5 Advanced Callback Configuration

- **Early Stopping**: The training was configured to stop if the validation loss did not improve for 3 consecutive epochs, preventing the model from training beyond its optimal point.
- **ReduceLROnPlateau**: This callback automatically reduces the learning rate by a factor of 0.5 if the validation loss plateaus for 2 epochs, which helps the model fine-tune its weights more effectively.

3.9 Model Training Implementation

3.9.1 Batch Processing with Class Weights

The training process incorporates weighted batch sampling to maintain balanced learning:

Batch Processing Strategy:

- Batch Size: 64 samples per gradient update
- Weight Application: Class weights influence loss calculation per batch
- Memory Optimization: Balanced between computational efficiency and convergence stability

3.9.2 Multi-Metric Monitoring System

Comprehensive performance tracking through multiple evaluation metrics:

Primary Monitoring Metrics:

- **Training Loss:** measures fit to the training data and is the primary optimization objective (when using focal loss this is replaced/augmented accordingly).
- **Validation Loss:** monitors generalization; used by early stopping and learning-rate schedulers to detect overfitting.
- **Accuracy:** overall proportion of correct predictions; useful as a broad summary but insensitive to class imbalance.
- **Precision:** proportion of predicted positives that are true positives; critical for assessing false-alarm (FP) behavior on attack classes.
- **Recall:** proportion of true positives detected; a primary safety metric for intrusion detection (minimizing missed attacks).
- **F1-Score:** harmonic mean of precision and recall; provides a single balanced measure for model ranking when both precision and recall matter.
- **FPR:** proportion of benign samples misclassified as attacks — key for measuring nuisance/alert fatigue in operational settings.
- **FNR:** proportion of attacks missed by the system — constrained to remain below the operational threshold (e.g., $\leq 1\%$) in tuning.

3.9.3 Automatic Model Checkpointing

Checkpoint Strategy:

- Best Model Preservation: Only saves improvements in validation performance
- Complete Model Save: Architecture + weights for full reproducibility
- Recovery Capability: Enables training resumption from optimal state

3.10 Comprehensive Model Evaluation

3.10.1 Baseline Model Evaluation

Before optimization, a baseline model was trained using standard Bi-LSTM parameters without Bayesian tuning, focal loss, or resampling. This establishes a comparative foundation for measuring improvements achieved by the proposed optimized framework.

- Test Set Evaluation

Table 3.5: Test Set Evaluation

Class (Label)	Precision	Recall	F1-Score	FNR	FPR	Support
Backdoor (0)	0.79	1.00	0.88	0.0000	0.0129	3,997
DDoS (1)	0.91	0.97	0.94	0.0270	0.0045	3,998
DoS (2)	0.90	0.97	0.93	0.0348	0.0053	4,000
Injection (3)	0.88	0.98	0.93	0.0210	0.0063	3,999
MITM (4)	0.21	0.97	0.34	0.0335	0.0088	209
Normal (5)	1.00	0.93	0.96	0.0662	0.0056	59,991
Password (6)	0.88	1.00	0.93	0.0038	0.0065	3,998
Scanning (7)	0.93	0.99	0.96	0.0068	0.0034	3,999
XSS (8)	0.90	0.93	0.92	0.0663	0.0049	3,999

- Confusion Matrix

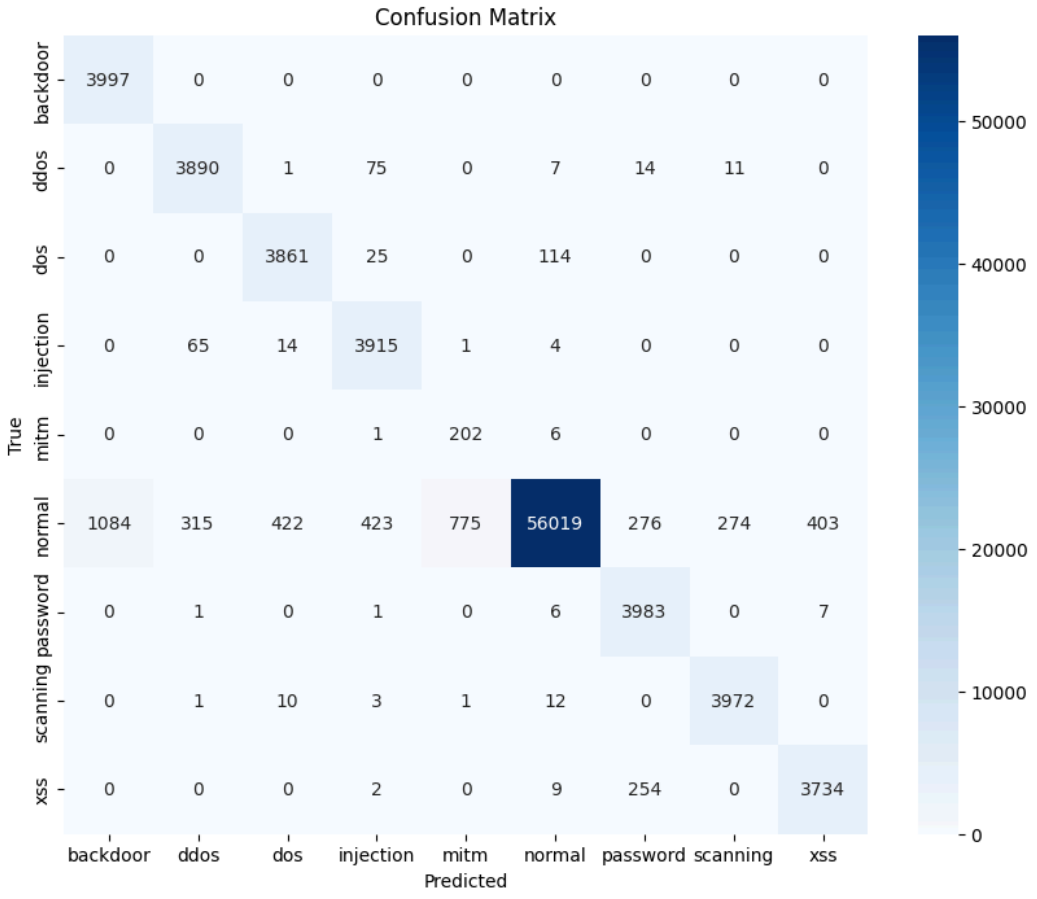


Fig. 3.3: Baseline Model Confusion Matrix

- Training and Validation Loss and Accuracy

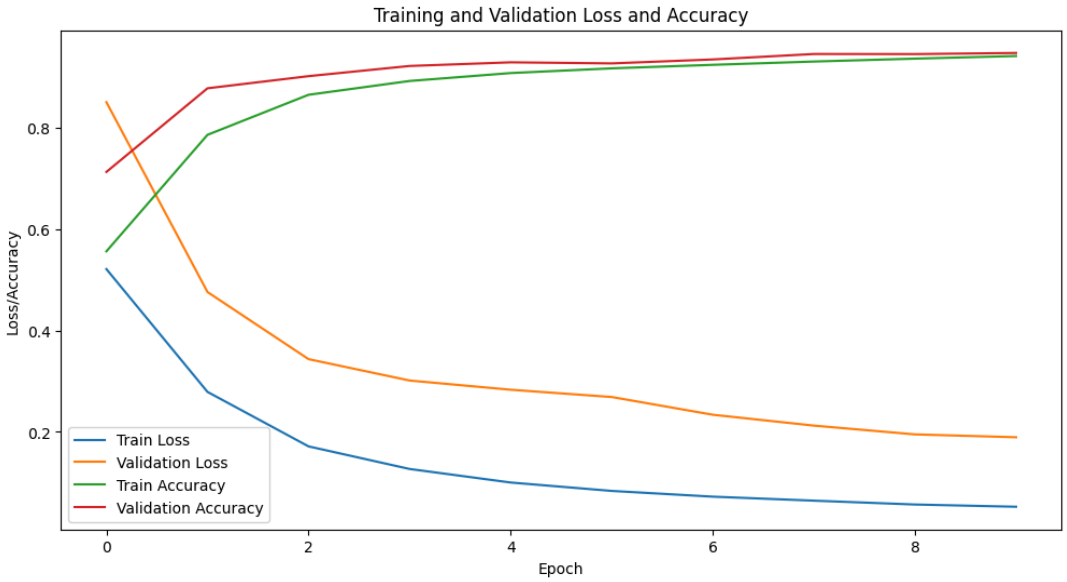


Fig. 3.4: Baseline Model Training and validation Loss and Accuracy

3.10.2 Performance Validation

- Test Set Evaluation

The model was evaluated on the held-out test set (88,200 samples). The overall test performance is very high (accuracy ≈ 0.99) and the weighted averages indicate consistently strong behaviour across most classes (weighted precision ≈ 0.99, weighted recall ≈ 0.99, weighted F1 ≈ 0.99). Table 3.10.1.1 summarizes per-class metrics (precision, recall, F1, and support) together with derived confusion counts (true positives, false negatives, false positives and predicted positives).

Table 3.6: Test Set Evaluation

Class (encoded)	**Support**	**Precision**	**Recall**	**F1**	**TP**	**FN**	**FP**	**Predicted positives (TP+FP)**	**FPR**	**FNR**
backdoor (0)	3,999	1.00	1.00	1.00	3,999	0	0	3,999	0.000000	0.000000
ddos (1)	3,999	0.98	0.98	0.98	3,919	80	78	3,997	0.000926	0.015754
dos (2)	4,000	0.96	0.97	0.96	3,880	120	168	4,048	0.001995	0.033000
injection (3)	3,999	0.97	0.98	0.97	3,919	80	118	4,037	0.001401	0.024506
mitm (4)	209	0.48	0.97	0.64	203	6	222	425	0.002523	0.033493
normal (5)	59,995	1.00	0.99	0.99	59,395	600	208	59,603	0.007375	0.010984
password (6)	3,999	0.93	0.99	0.96	3,959	40	318	4,277	0.003777	0.005751

scanning (7)	4,000	0.98	1.00	0.99	4,000	0	64	4,064	0.000760	0.004250
xss (8)	4,000	0.98	0.93	0.95	3,720	280	95	3,815	0.001128	0.068000

- Confusion Matrix Analysis

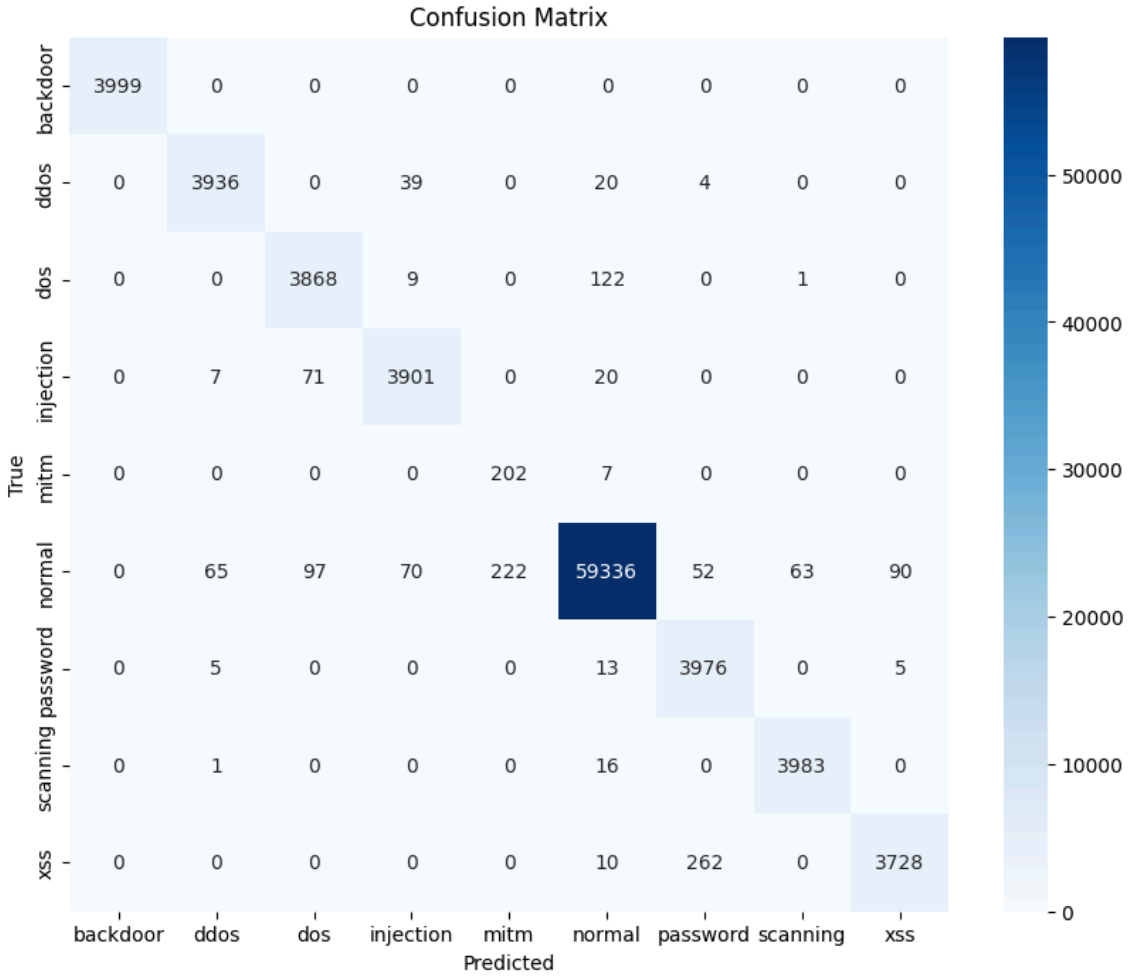


Fig. 3.3: Confusion Matrix Before Hyperparameter Tuning

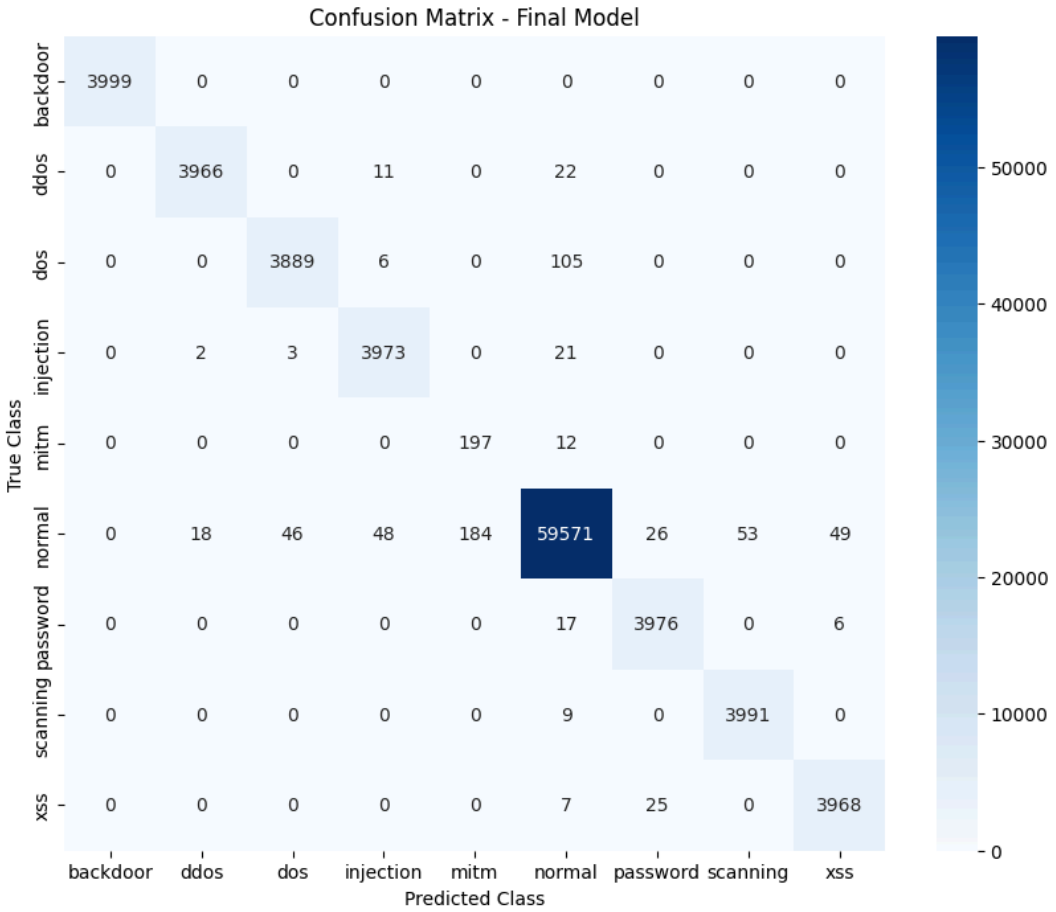


Fig. 3.4: Confusion Matrix After Hyperparameter Tuning

- Training and Validation Loss and Accuracy

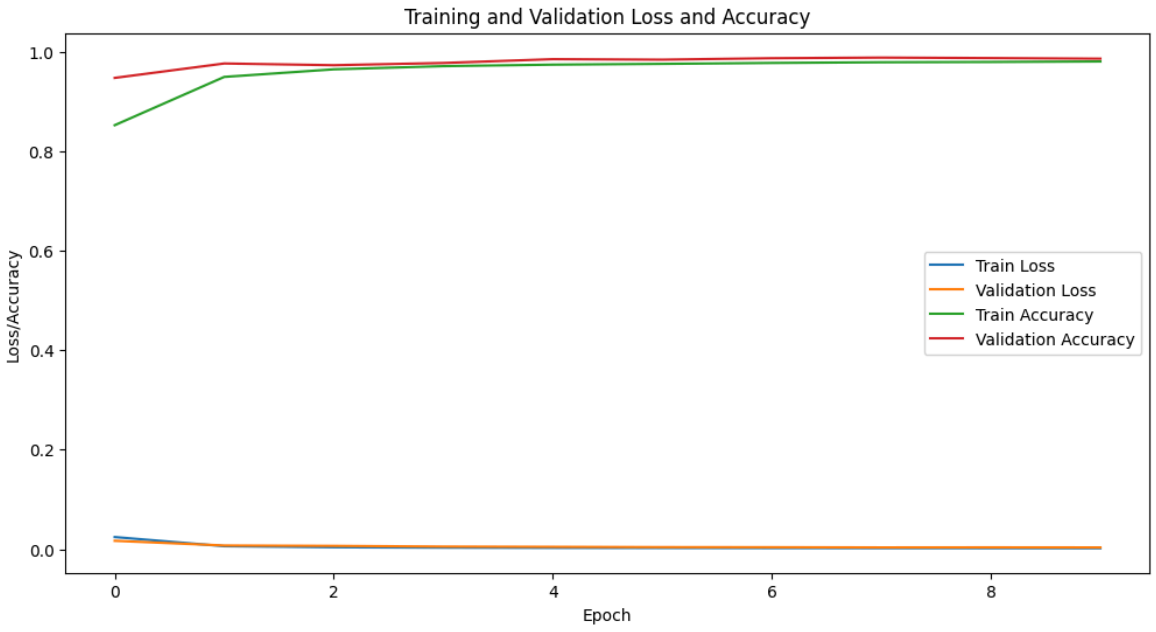


Fig. 3.5: Training and Validation Loss and Accuracy Before Hyperparameter Tuning

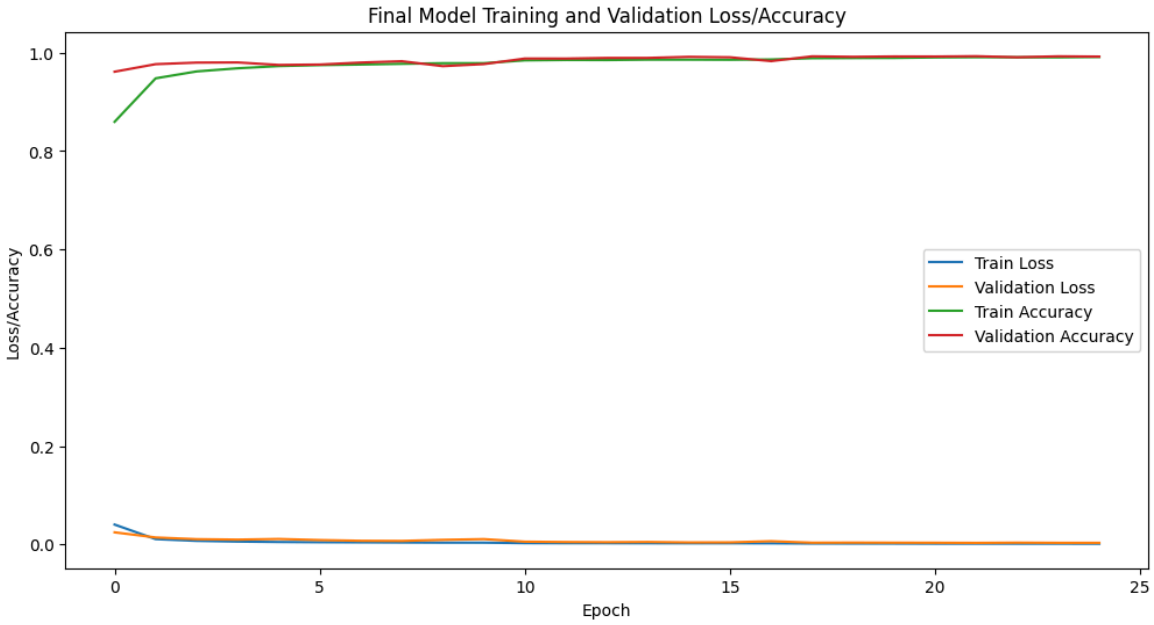


Fig. 3.6: Training and Validation Loss and Accuracy After Hyperparameter Tuning

3.10.3 Model Selection Criteria

The model selection process employed a comprehensive multi-faceted evaluation strategy to identify the optimal Bi-LSTM architecture for network intrusion detection. The selection was guided by the following rigorous criteria:

Primary Performance Thresholds:

- **Validation Accuracy**: > 90% (minimum threshold for practical deployment)
- **False Positive Rate (FPR)**: ≤0.75% (critical for minimizing false alarms in operational environments)
- **False Negative Rate (FNR)**: ≤1.0% (essential for ensuring comprehensive threat detection)
- **Validation AUC**: > 0.99 (ensuring robust class separation capability)

REFERENCE

[1] Henry, A., Gautam, S., Khanna, S., Rabie, K., Shongwe, T., Bhattacharya, P., Sharma, B., & Chowdhury, S. (2023). Composition of Hybrid Deep Learning Model and Feature Optimization for Intrusion Detection System. Sensors, 23(2), 890. https://doi.org/10.3390/s23020890

[2] B. Rana and R. Singh, "IoT: False Positive Rate Reduction in Intrusion Detection Systems," International Journal of Research in Engineering and Science (IJRES), vol. 10, no. 7, pp. 72-79, July 2022.

[3] J. Mijalkovic and A. Spognardi, "Reducing the False Negative Rate in Deep Learning Based Network Intrusion Detection Systems," Algorithms, vol. 15, no. 8, p. 258, 2022. doi: 10.3390/a15080258.

[4] A. Alabbadi and F. Bajaber, "An Intrusion Detection System over the IoT Data Streams Using eXplainable Artificial Intelligence (XAI)," Sensors, vol. 25, no. 3, p. 847, 2025. doi: 10.3390/s25030847.

[5] M. Siganos, P. Radoglou-Grammatikis, I. Kotsiuba, E. Markakis, I. Moscholios, S. Goudos, and P. Sarigiannidis, "Explainable AI-based Intrusion Detection in the Internet of Things," in Proc. 18th Int. Conf. Availability, Reliability and Security (ARES), Benevento, Italy, Aug.–Sept. 2023, pp. 1–10. doi: 10.1145/3600160.3605162.

[6] M. Saied and S. Guirguis, "Explainable artificial intelligence for botnet detection in Internet of Things," Scientific Reports, vol. 15, p. 7632, 2025. doi: 10.1038/s41598-025-90420-6.

[7] S. J. Moore, F. Cruciani, C. D. Nugent, S. Zhang, I. Cleland, and S. Sani, "Deep learning for network intrusion: A hierarchical approach to reduce false alarms," *Intelligent Systems with Applications*, vol. 18, p. 200215, 2023. doi: 10.1016/j.iswa.2023.200215.

[8] S. Sivamohan and S. S. Sridhar, "An optimized model for network intrusion detection systems in Industry 4.0 using XAI based Bi-LSTM framework," Neural Computing and Applications, vol. 35, 2023, doi: 10.1007/s00521-023-08319-0.

[9] N. Moustafa, "ToN_IoT Datasets," UNSW, 2020. [online]. Available: https://research.unsw.edu.au/projects/toniot-datasets. [Accessed: Feb. 14, 2025].

[10] M. Al-kahtani, Z. Mehmood, T. Sadad, I. Zada, G. Ali, and M. El Affendi, "Intrusion detection in the Internet of Things using fusion of GRU-LSTM deep learning model," Intelligent Automation and Soft Computing, vol. 37, 2023, doi: 10.32604/iasc.2023.037673.

[11] A. R. Gad, M. Haggag, A. A. Nashat, and T. M. Barakat, “A distributed intrusion detection system using machine learning for IoT based on ToN-IoT dataset,” International Journal of Advanced Computer Science and Applications (IJACSA), vol. 13, no. 6, pp. 548–556, 2022.

[12] N. W. Khan, M. S. Alshehri, M. A. Khan, S. Almakdi, N. Moradpoor, A. Alazeb, S. Ullah, N. Naz, and J. Ahmad, “A hybrid deep learning-based intrusion detection system for IoT networks,” Mathematical Biosciences and Engineering, vol. 20, no. 8, pp. 13491–13520, 2023.

[13] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Federated Learning for IoT Intrusion Detection," *AI*, vol. 4, no. 3, pp. 1–25, Jul. 2023, doi: 10.3390/ai4030038.